

AI Harness 系統設計開發日誌

一、與 AI 的互動紀錄 (Prompt / Chat History)

在本次「自主科學文獻研究助理 (Autonomous Science Research Agent)」系統設計過程中，我透過與 AI 助理的多次對話來收斂需求與迭代架構。以下為關鍵的互動 Prompt 紀錄：

- **探索與收斂階段：**
 - *Prompt 1:* 「這個作業我要怎麼做？」——釐清作業核心要求，確認重點在於「系統設計 (System Design) 思維」而非模型訓練或演算法推導。
 - *Prompt 2:* 「你有沒有建議的題目？」——請 AI 結合大型語言模型 (LLM) 與 Agent 最新發展趨勢，發想符合「至少 3 個 tools」與「多步工作流」要求的題目。最後選定「自主科學文獻研究助理」。
- **系統設計階段：**
 - *Prompt 3:* 「我要這個題目:自主科學文獻研究助理... 需完成一個 AI Harness 系統設計。包含問題定義、架構、tool usage 機制、agent workflow 等... 依次給我內容」——建立系統核心邏輯與架構梳理。
 - *Prompt 4:* 「根據剛剛上面的回覆，先生成一個書面報告(必繳)。」——將系統邏輯轉換為正式的 2-5 頁書面報告格式。
- **圖表與視覺化階段：**
 - *Prompt 5:* 「需以視覺化方式呈現系統設計，內容包含 AI system architecture、orchestration / workflow flow...」——探討如何繪製 Sequence Diagram 與架構圖，並利用 AI 輔助生成資訊圖表 (Infographic)。
 - *Prompt 6:* 「請幫我草擬最後一份必交文件 log.md 的內容」——總結開發過程，完成最後一項交付文件。

二、系統設計迭代過程 (System Design Iteration)

本系統的設計並非一步到位，而是經過多次與 AI 探討、反思系統缺陷後的逐步迭代：

1. **Q (架構挑戰)：**最初的想法是直接串接學術搜尋 API，將結果直接丟給使用者 (V1 基礎檢索版)，但這似乎缺乏 Agent 的自主性與多步推理能力？
1. **A (設計迭代)：**沒錯。如果在 V1 就停止，這只是一個傳統的搜尋腳本。進一步測試後發現，單一工具處理複雜研究會消耗過多 Token 且極易混淆。因此迭代出 **V2「工具解耦版」**，將功能精準拆分為 `search_academic_papers` (僅抓 ID)、`extract_paper_details` (抓摘要與貢獻) 與 `cross_reference_analysis` (表格化比對)，有效節省 Token 並避免 LLM 上下文溢位。
2. **Q (流程升級)：**工具解耦後，系統如何記住過去的搜尋進度並做出動態決策？
2. **A (設計迭代)：**這催生了最終的 **V3「記憶體與自主決策版」**。為了具備真實「研究助理」的特性，我們加入了短/長期記憶體 (Memory System) 儲存向量化特徵，並全面改採 ReAct (Reasoning and Acting) 循環機制，使系統具備動態調整搜尋策略的自主能力。

三、架構調整與設計決策 (Architecture & Design Decisions)

在底層架構的設計上，我針對穩定性與效能做出了以下關鍵的技術決策探討：

1. **Q (決策探討)：**為什麼在流程控制上選擇 ReAct 模式，而不是開發上較簡單的固定 Pipeline Chain？
1. **A (架構決策)：**因為學術研究是一個高度動態的過程。如果搜尋 API 回傳 0 篇文獻，固定的 Pipeline 會直接報錯中斷。改用 ReAct 模式後，LLM 可

以在 Observation 階段發現「查無資料」，並透過下一輪的 Thought 自主決定更換關鍵字或放寬搜尋條件，大幅提高了系統的彈性。

2. **Q (穩定性探討):** 但 ReAct 這種高度自由的循環，會不會導致 Agent 因為找不到特定文獻而陷入「無限檢索」的死迴圈？
2. **A (架構決策):** 這是非常核心的風險。因此，我在 Orchestration 的工作流中強制加入了「Max Iterations = 10」的系統級中斷防呆機制。只要達到迴圈上限，系統就會強制終止檢索，並統整現有資訊輸出報告，確保系統不會因為無限消耗 Token 而崩潰。
3. **Q (評量探討):** AI Agent 處理學術文獻很容易出現「幻覺 (捏造不存在的論文)」，傳統的 Accuracy 或 Loss 根本無法評估，該如何衡量系統好壞？
3. **A (架構決策):** 這正是我們放棄傳統機器學習指標的原因。本設計決策引入「LLM-as-a-Judge」機制與「幻覺率 (Hallucination Rate)」指標，不看 Loss，而是直接驗證 Agent 最終報告引用的文獻是否真實存在，以及是否忠於原著。

四、問題分析與修正過程 (Problem Analysis & Fixes)

在模擬實際運作時，發現了兩個潛在的嚴重問題，並進行了機制上的修正：

1. **Q (效能瓶頸分析):** 如果 Agent 為了深入研究，一次性抓取了 5-10 篇學術論文的 PDF 全文，極容易引發 LLM 的上下文溢位 (Context Window Overflow)，這部分如何克服？
1. **A (修正過程):** 經過日誌分析，論文全文包含太多實驗數據與公式贅述，不僅消耗運算資源還會造成嚴重的資訊干擾。因此我緊急修正了 Tool 2 (extract_paper_details) 的設計，限制其 僅抓取「摘要 (Abstract)」、「關鍵字」與「主要貢獻」；同時在 Tool 1 的參數中嚴格設定 max_results = 5，從資料源頭進行控制。

2. **Q (例外處理分析):** 外部學術資料庫 (如 ArXiv API) 偶爾會發生 Timeout 或拒絕連線的情況, API 報錯時, 系統會不會跟著當機停擺?
2. **A (修正過程):** 早期版本的確會因為 API 拋出 Exception 而崩潰。後來的修正方案是加入了 **錯誤轉化機制**: 當 API 發生錯誤時, 系統不直接中斷, 而是將錯誤代碼轉化為自然語言的 Observation (例如: "API Timeout, please try again or change tool") 回傳給 LLM。透過這種方式, Agent 能自主決定要等待重試還是換個工具, 大幅提升了系統的「錯誤恢復率 (Error Recovery Rate)」